

Trabalho Final de Segurança Computacional

Implementação Didática do Protocolo Kerberos

Disciplina: Segurança Computacional - CIC0201

Instituição: Universidade de Brasília

Alunos:

Henrique Luz Martins - 200061691

Gustavo Choueiri - 232014010

Relatorio Tecnico - Kerberos Didatico

1. Objetivo

Este trabalho implementa uma versao didatica do protocolo Kerberos em Python.

O objetivo foi demonstrar, na pratica, como um cliente pode se autenticar em um servico protegido sem enviar sua senha pela rede, usando um Servidor de Autenticacao, um Ticket Granting Server, tickets criptografados, chaves de sessao e authenticators.

A aplicacao final escolhida foi um servico simples de notas. A simplicidade do servico foi proposital, pois o foco principal do trabalho e o fluxo de autenticacao.

2. Arquitetura

A implementacao foi dividida em quatro componentes principais:

- Cliente Kerberos
- Servidor de Autenticacao
- Ticket Granting Server
- Servico protegido de notas

O Servidor de Autenticacao emite o TGT. O Ticket Granting Server valida o TGT e emite um ticket especifico para o servico. O servico protegido valida o ticket recebido, valida o authenticator do cliente e responde de forma criptografada.

3. Fluxo implementado

O fluxo implementado segue a ideia central do Kerberos:

1. O cliente deriva sua chave localmente a partir da senha.
2. O cliente envia uma AS_REQ ao Servidor de Autenticacao.
3. O AS valida o usuario, verifica o timestamp da requisicao e emite um TGT.
4. O cliente envia o TGT ao TGS junto com um authenticator.
5. O TGS valida o TGT, valida o authenticator e emite um ticket de servico.
6. O cliente envia o ticket de servico ao servico protegido junto com outro authenticator.

7. O servico valida o ticket, valida o authenticator e processa a requisicao.
8. O servico responde criptografado com a chave de sessao Cliente-Servico.
9. O cliente confirma a autenticacao mutua verificando o valor timestamp + 1.

4. Criptografia utilizada

A implementacao usa criptografia simetrica com AES-GCM.

O AES-GCM foi escolhido porque fornece confidencialidade e autenticidade da mensagem. Assim, se uma mensagem criptografada for alterada, a descriptografia falha.

As chaves dos usuarios sao derivadas a partir da senha com PBKDF2-HMAC-SHA256. Dessa forma, a senha nao precisa ser usada diretamente como chave criptografica.

Tambem foram usadas funcoes auxiliares para gerar chaves aleatorias de sessao e converter chaves para texto quando elas precisam ser colocadas dentro de tickets criptografados.

5. Tickets e chaves de sessao

O AS emite uma chave de sessao Cliente-TGS e um TGT.

O TGT contem:

- identificacao do cliente;
- identificacao do TGS;
- chave de sessao Cliente-TGS;
- horario de emissao;
- horario de expiracao.

O TGT e criptografado com a chave compartilhada entre AS e TGS. Por isso, o cliente recebe o TGT, mas nao consegue ler nem alterar seu conteudo.

Depois, o TGS emite uma chave de sessao Cliente-Servico e um ticket de servico.

O ticket de servico contem:

- identificacao do cliente;
- identificacao do servico;
- chave de sessao Cliente-Servico;
- horario de emissao;
- horario de expiracao.

Esse ticket e criptografado com a chave do servico protegido.

6. Authenticators e protecao contra replay

A implementacao usa authenticators com timestamp nas requisicoes ao TGS e ao servico protegido.

O authenticator serve para provar que o cliente conhece a chave de sessao correta e que a requisicao e recente.

Alem da verificacao de timestamp, foram adicionadas caches de authenticators ja utilizados no TGS e no servico. Com isso, se uma mesma requisicao for reenviada, ela e rejeitada como possivel tentativa de replay.

Essa protecao foi implementada de forma didatica, em memoria, durante a execucao do servidor.

7. Autenticacao mutua

O servico protegido tambem autentica sua resposta para o cliente.

Depois de validar o ticket e o authenticator, o servico responde com uma mensagem criptografada usando a chave de sessao Cliente-Servico. Dentro dessa resposta, ele envia o valor timestamp + 1.

O cliente verifica esse valor. Se ele estiver correto, o cliente confirma que a resposta veio de uma entidade que conhece a chave de sessao correta.

8. Servico protegido

O servico escolhido foi um sistema simples de notas.

O cliente pode:

- adicionar uma nota;
- listar as notas cadastradas.

A acao solicitada e o conteudo da nota sao enviados dentro de uma requisicao criptografada. Assim, alem do ticket e do authenticator, o proprio pedido feito ao servico tambem fica protegido pela chave de sessao Cliente-Servico.

9. Testes

Foi criado o arquivo test_kerberos.py para validar automaticamente o projeto.

O teste verifica:

- derivacao de chave por senha;
- rejeicao de senha incorreta;
- emissao do TGT pelo AS;
- emissao do ticket de servico pelo TGS;
- acesso ao servico protegido;
- autenticacao mutua;
- bloqueio de replay no TGS;
- bloqueio de replay no servico protegido.

O teste automatico ajuda a demonstrar que o fluxo principal do protocolo esta funcionando e que as protecoes adicionadas nao quebraram a comunicacao normal entre cliente, AS, TGS e servico.

10. Dificuldades encontradas

Uma das principais dificuldades foi organizar a troca de mensagens entre os componentes sem usar uma biblioteca pronta de Kerberos.

Tambem foi necessario separar corretamente o que cada entidade pode ler. Por exemplo, o cliente recebe o TGT, mas nao deve conseguir abrir seu conteudo. O mesmo vale para o ticket de servico, que deve ser lido apenas pelo servico protegido.

Outra dificuldade foi implementar a protecao contra replay. Apenas verificar se o timestamp e recente nao impede que uma mesma mensagem seja reenviada dentro da janela de tempo permitida. Por isso, foi adicionada uma verificacao de authenticators ja utilizados.

11. Limitacoes

Esta implementacao e didatica e nao deve ser tratada como uma implementacao de Kerberos pronta para producao.

Algumas limitacoes sao:

- os usuarios de demonstracao ficam definidos no codigo;
- as chaves internas tambem sao definidas no codigo;
- a cache de replay fica apenas em memoria;
- nao ha persistencia real de usuarios;
- nao ha TLS;
- nao ha integracao com um KDC real;
- nao ha suporte a multiplos servicos dinamicos.

Essas escolhas foram feitas para manter o projeto simples, executavel e adequado ao objetivo academico.

12. Conclusao

O projeto implementa os principais conceitos do Kerberos de forma didatica: autenticacao inicial, emissao de TGT, emissao de ticket de servico, uso de chaves de sessao, validacao de authenticators, autenticacao mutua e protecao contra replay.

A implementacao tambem inclui um teste automatico que executa o fluxo completo e valida os principais requisitos de seguranca demonstrados no trabalho.